

192분류 도출 알고리즘 — Backend Encoding Logic

5축 곱연산: 16(Bauman) × 3(UV) × 4(Need) = 192. 각 축의 점수 → 분기점 → 최종 코드 결정

단계	로직 설명	예시 / 결과
STEP 1: 원시 점수 계산 (Raw Score)		
각 문항 응답 → '가중치 & 점수 계산' 시트 로직대로 6축 점수 누적		O_score, D_score, S_score, P_score, W_score, T_score, UV_LT/MD/DP, HY/BR/AA/TC
STEP 2: O/D 결정 — Bauman 1번째 글자		
O_score >= 35 AND O_score > D_score → 'O' (지성) D_score >= 35 AND D_score > O_score → 'D' (건설) 둘 다 낮음 → Q10 자가진단으로 결정 둘 다 높음 (복합) → 더 높은 쪽 우선 + 'COMBO' 태그		예: O=50, D=20 → 'O'
STEP 3: S/R 결정 — Bauman 2번째 글자		
S_score 총점 (Q1+Q3+Q4+Q5+Q8+Q9+Q11+Q36) >= 30 → 'S' (민감) Q9에서 ①~⑥ 선택 시 강제로 'S' S_score < 30 → 'R' (저항)		예: S=45 → 'S'
STEP 4: P/N 결정 — Bauman 3번째 글자		
P_score 총점 (Q6+Q7+Q8+Q12+Q13+Q14+Q21) >= 25 → 'P' (색소성) P_score < 25 → 'N' (비색소)		예: P=40 → 'P'
STEP 5: W/T 결정 — Bauman 4번째 글자		
W_score 총점 (Q6+Q7+Q8+Q15+Q16+Q17) >= 25 → 'W' (주름형) W_score < 25 → 'T' (탄력형)		예: W=30 → 'W'
STEP 6: UV 그룹 결정 — Light/Medium/Deep		
Q18 응답 직접 매핑: ① → LT / ② → MD / ③ → DP Q19 (모발색) 가산점으로 ±1단계 보정 거주지 위도로 자동 보정 (≥40° → LT 보정 / ≤25° → DP 보정)		예: Q18=③, Q19=②, 위도=37° → MD
STEP 7: Need 결정 — HY/BR/AA/TC		
1순위 (Q6, 가중치 60) + 2순위 (Q7, 가중치 30) + 현재 사용 (Q29-Q30, 가중치 10) 동일 카테고리 → 해당 Need 다른 카테고리 → TC (토탈케어) 점수 분산 50% 이상 → TC		예: Q6④(BR×60), Q7④(BR×30) → BR
STEP 8: 최종 코드 조립		
Bauman(4글자) + '-' + UV(2글자) + '-' + Need(2글자) → 192분류 중 1개		예: OSPW-MD-AA
STEP 9: Modifier·Filter 적용		
Modifier (Q5, Q22, Q23, Q27) → 결과 화면 캐비닛·재진단 알림 Filter (Q9, Q38, Q39, Q40) → 추천 성분 자동 제외		결과 화면 표시
STEP 10: 신뢰도 계산		
Q1+Q11, Q4+Q10 응답 일치도 → confidence score (0~100%) < 70% → '재진단 권장' 플래그 + 전문가 1:1 유도		예: confidence = 92%

```

def classify_skin_type(responses: dict) -> dict:
    # STEP 1: 원시 점수 계산
    scores = compute_raw_scores(responses) # {O, D, S, P, W, T, LT, MD, DP, HY, BR, AA, TC}

    # STEP 2: Bauman O/D
    if responses['Q9'] in [1,2,3,4,5,6]: # 질환 진단/의심 → 안전모드
        skin_filter = 'SAFE_MODE'
    od = 'O' if scores['O'] >= 35 and scores['O'] > scores['D'] else 'D'

    # STEP 3: S/R
    if responses['Q9'] in [1,2,3,4,5,6]:
        sr = 'S' # 강제
    else:
        sr = 'S' if scores['S'] >= 30 else 'R'

    # STEP 4: P/N
    pn = 'P' if scores['P'] >= 25 else 'N'

    # STEP 5: W/T
    wt = 'W' if scores['W'] >= 25 else 'T'

    # STEP 6: UV
    uv_raw = {1:'LT', 2:'LT', 3:'MD', 4:'MD', 5:'DP', 6:'DP'}[responses['Q18']]
    uv = adjust_uv(uv_raw, responses['Q19'], responses['D3_latitude'])

    # STEP 7: Need
    need = decide_need(responses['Q6'], responses['Q7'], responses['Q29'], responses['Q30'])

    # STEP 8: 최종 코드
    bauman = od + sr + pn + wt
    final_code = f"{bauman}-{uv}-{need}"

    # STEP 9: Modifiers & Filters

```